

BEVSA: A Real-Time Bird's-Eye-View Semantic Segmentation Accelerator for Multi-Camera System

Sangho Lee^{*}, Jueung Jung^{*}, Wuyoung Jang^{*}, Jihyeon Hwang^{*}, and Kyuho Lee[†]

^{*}Graduate School of Artificial Intelligence / Department of Electrical Engineering, UNIST, Republic of Korea

[†]School of Electrical and Electronics Engineering, Yonsei University, Republic of Korea

lsh9901@unist.ac.kr / kyuho.jsn.lee@yonsei.ac.kr

Abstract— A bird's-eye-view (BEV) semantic segmentation accelerator (BEVSA) is proposed for real-time 3D space perception in multi-camera system (MCS). The view transformation from multi-camera-view to BEV obstructs the real-time operation on edge devices through 68.3 ms of time consumption during BEV pooling, which requires sorting and irregular memory access over wide searching space. Moreover, the 69.1% high average input activation sparsity of the segmentation process for the transformed BEV features results in excessive meaningless computations. For the real-time implementation of BEV semantic segmentation on edge platforms, this paper proposes two key features: 1) Block-decomposed hierarchical BEV pooling cluster that partitions the searching space in an MCS-suitable way and supports parallel pooling, achieving 43.2× speedup for BEV pooling over the edge computing platform; 2) Coarse-to-fine-grained zero skipping convolution cluster, which conducts coarse-grained zero skipping for all-zero channels and tile-wise fine-grained zero skipping, improving the convolution throughput by 1.61×. Implemented with 28nm technology, and evaluated on two representative MCS datasets, BEVSA finally achieves 23.1 frames-per-second of real-time BEV segmentation throughput with 167.4× higher energy-per-frame over the edge computing platform.

Keywords— 3D perception, BEV pooling, BEV semantic segmentation, heterogeneous cluster, multi-camera system, parallelization, sparsity exploitation.

I. INTRODUCTION & BACKGROUND

Emerging autonomous robots require a comprehensive understanding of wide 3D space for flawless interaction with the surrounding environment. Bird's-eye-view semantic segmentation (BEV-SS) plays a crucial role as an effective solution for 3D perception through a multi-camera system (MCS), which is widely deployed in autonomous driving. The bird's-eye-view (BEV) resolves the scale variation and occlusion challenges inherent in vision-based 3D perception by transforming the ego-view of each camera into an integrated view. It facilitates the expansion of the autonomous system to multi-camera settings for high accuracy, being well-suited for 3D perception of objects and road elements for subsequent functions, such as planning and control in autonomous driving [1]. Recently Lift-splat [2] was proposed for BEV-SS, which generates the depth-estimated camera features using a convolutional neural network (CNN), followed by rasterization through BEV pooling for view transformation. By minimizing the interference among cameras during its training for depth

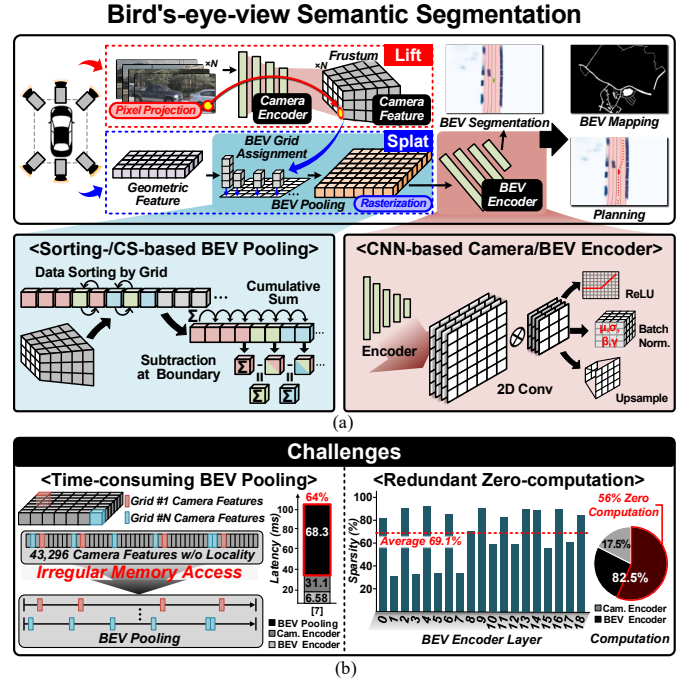
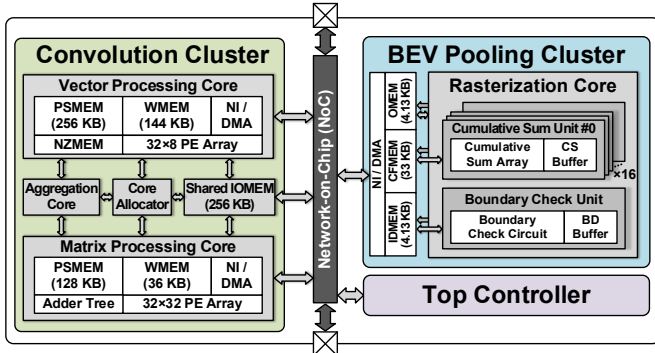


Fig. 1. Explanation of the bird's-eye-view semantic segmentation. (a) The entire process through lift and splat steps and (b) challenges of real-time implementation and motivation for hardware accelerator.

estimation, Lift-splat [2] recovers from partial damage of cameras in inference and maintains high accuracy, enabling robust view integration even with the various configurations of MCS. Therefore, it has been widely adopted in BEV applications [3]-[5].

Fig. 1(a) illustrates the overall process of BEV-SS [2] with MCS, consisting of two steps: *lift* and *splat*. During the *lift* step, a CNN-based camera encoder performs pixel-wise depth estimation on each 2D image to project into a 3D camera features as a frustum. Then, at the *splat* step, the features in each frustum are assigned to the nearest BEV grid according to the corresponding geometric features. The assigned features are rasterized through BEV pooling within each grid, adopting cumulative sum (CS). After sorting the data according to its grid, CS is applied across the entire feature, followed by subtraction at each grid boundary. Afterward, the CNN-based BEV encoder takes the rasterized BEV features as input to produce segmentation outputs.



PSMEM: Partial-sum Memory, WMEM: Weight Memory, NZMEM: Nonzero Memory, IOMEM: I/O Memory, BD: Boundary IDMEM: Grid ID Memory, CFMEM: Camera Feature Memory, OMEM: Output Memory

Fig. 2. Overall architecture of the proposed BEVSA.

To ensure the safety of autonomous driving, satisfying the real-time constraint is imperative, at least 12 Hz (83.3 ms/frame) according to [6]. However, real-time implementation of BEV-SS on an edge computing platform [7] is unfeasible for two reasons, as illustrated in Fig. 1(b). First, the rasterization process utilizes time-consuming BEV pooling. Due to the irregular memory access in BEV pooling induced by extracting features with the same grid ID among 43,296 camera features [8], the BEV pooling accounts for 64% of total latency. Additionally, the BEV encoder, which accounts for 82.5% of convolution operations, incurs substantial computational load with redundant zero computations. The input of each layer in the BEV encoder exhibits an average sparsity of 69.1%, resulting from the sparse BEV features and ReLU activation function. Therefore, the redundant zero-computations constitute 55.7% of total convolution operations across the two encoders.

In this paper, a real-time BEV semantic segmentation accelerator, named BEVSA, for outdoor MCS is proposed. BEVSA is designed with a heterogeneous cluster architecture, facilitating parallel BEV pooling and zero skipping optimized for BEV-SS through the following SW-HW co-optimizations:

- A block-decomposed BEV pooling method is proposed for removing redundant searching operations and inter-block parallel BEV pooling without sorting.
- A hierarchical BEV pooling cluster (BPC) is designed for further reduction of pooling searching space by 87.5% and supporting intra-block parallelism of BEV pooling.
- A coarse-grained zero skipping method with on-the-fly pruning is introduced for 50.1% reduced computation in the BEV encoder by permanently removing meaningless channels.
- A fine-grained zero skipping method on hybrid sparse-dense convolution cluster (CC) is proposed for efficient CNN inference utilizing sparsity prediction through inter-frame similarity.

BEVSA is designed in RTL Verilog and implemented in 28nm CMOS technology to demonstrate its feasibility. $2.45\times$ and $2.33\times$ throughput improvements are achieved compared with baseline [7] for BEV-SS with [2] on nuScenes [6] and Lyft Level 5 [9] datasets. The rest of the paper is organized as

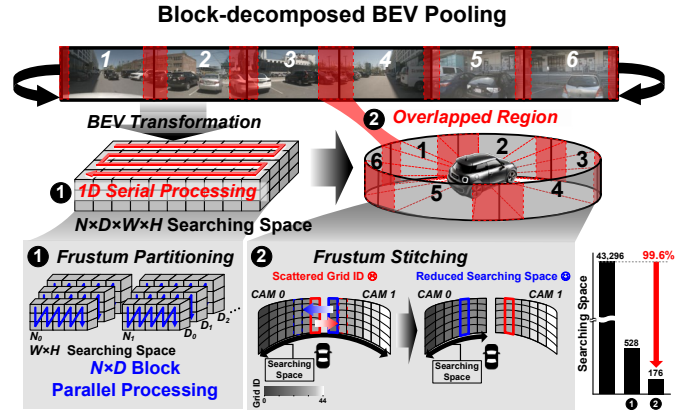


Fig. 3. The proposed frustum partitioning and stitching of block-decomposed BEV pooling for searching space reduction.

follows. Section II introduces the overall architecture of the proposed BEVSA. Sections III and IV describe the hierarchical BPC with block-based algorithm optimization and the coarse-to-fine-grained zero skipping, supported by hybrid sparse-dense CC, respectively. Section V shows the evaluation results, followed by the conclusion in Section VI.

II. OVERALL ARCHITECTURE

Fig. 2 illustrates the overall architecture of the proposed BEVSA. It comprises a CC for the camera encoder and BEV encoder, as well as a BPC for BEV pooling and a top controller interconnected through the network-on-chip (NoC). To facilitate efficient convolution processing for inputs with various sparsity of BEV encoder, the basis of CC rises in the hybrid architecture of [10] for sparsity exploitation. It consists of a vector processing core (VPC) for high-sparsity input and a matrix processing core (MPC) for dense input, with each employing 32×8 processing element (PE) array and 32×32 PE array, respectively. The aggregation core, core allocator, and 256 KB shared I/O memory are included between the pair of VPC and MPC. The BPC comprises a rasterization core, a camera feature memory, a grid ID memory, and an output memory. The rasterization core contains 16 CS units with the CS array and a boundary check (BC) unit with the BC circuit. The CS array iteratively accumulates input features, and the results are stored in the CS buffer. The boundary information obtained from the BC circuit of the BC unit is utilized for CS trick [2], which performs subtraction between boundary CS results for BEV pooling. Consequently, only the subtracted results corresponding to boundaries are stored in output memory with their grid IDs.

III. BLOCK-DECOMPOSED HIERARCHICAL BEV POOLING CLUSTER

A. Block-Decomposed BEV Pooling

Irregular memory access is the most crucial factor in latency during the BEV pooling process, which is caused by sorting and BC over a wide searching space based on multiple camera features. To reduce the extensive searching space, previous 3D perception accelerators [11] and [12] divided input points into

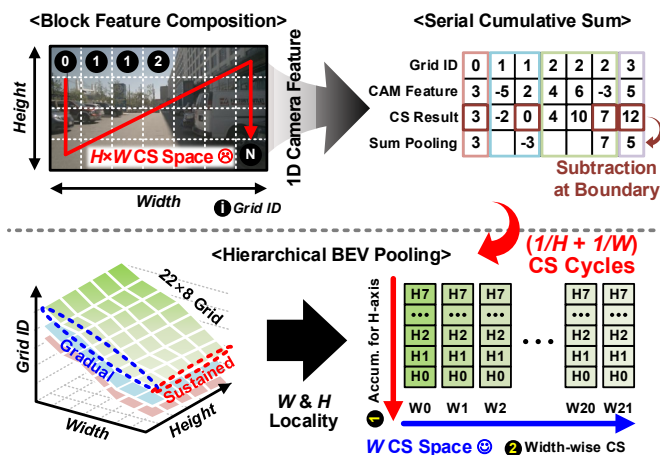


Fig. 4. The operation of hierarchical BEV pooling within a block based on grid ID locality across dimensions.

distinct spaces. Reference [11] presented a *window-search algorithm* for monocular RGB-D data, while [12] proposed a *spherical-coordinate-based bin searching* for LiDAR data. However, none of them dealt with the overlapped regions between sensors because they leveraged only a single sensor. In MCS, overlapping regions among the sensors must be considered when partitioning the space, as illustrated in Fig. 3. This requirement hinders the direct adoption of previous single-sensor-based space partitioning methods. Therefore, this paper proposes a block-based BEV, which divides the MCS inputs while handling overlapping regions between multiple images.

The bottom of Fig. 3 illustrates the proposed block-decomposed BEV pooling, which reduces the searching space by utilizing frustum partitioning based on the grid ID locality across dimensions. After the frustum partitioning on each camera and depth dimensions in 1D camera features, grid IDs appear ordered within each block, enabling CS to be performed without additional sorting. Each partitioned block can be processed in parallel within a searching space reduced by $82\times$ for locating and accumulating the features of a specific grid ID.

However, duplicated searching space occurs as grid IDs are partitioned across distinct blocks, which necessitates accessing camera features of neighboring blocks. The proposed frustum stitching eliminates redundant searches by reorganizing the features of each camera boundary. The overlapping regions are gathered based on the width of neighboring camera blocks, sorted, and then reallocated according to the grid ID distribution of each block. Through frustum partitioning and stitching, the searching space of BEV pooling is reduced by 99.6%, enabling flawless parallelization across each block.

B. Hierarchical BEV Pooling Cluster

Fig. 4 illustrates the hierarchical BEV pooling process within each block. The conventional CS trick requires serial computation on camera features encoded along the number of height (H) and width (W). This requires $H \times W$ registers to store the CS results along with the BC for the maximum grid IDs. On the other hand, the proposed hierarchical BPC enhances the computational parallelism by leveraging the locality of grid IDs with respect to the height and width. The grid IDs within a block exhibit a linear variation along the width axis while remaining

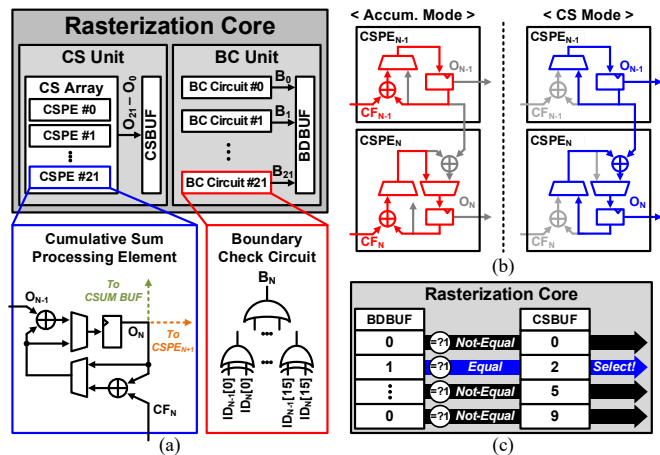


Fig. 5. (a) The architecture of cumulative sum processing element (CSPE) and boundary check circuit (BCC) in CS unit and BC unit, (b) two operating modes of CSPE, and (c) the CS result storage flow based on boundary condition.

constant along the height axis. Since variation in height has a minimal impact on the BEV grid ID, width-wise parallel accumulation of camera features is performed H times without grid ID comparison with only W registers. Subsequently, the CS is applied along the width axis during W cycles. Therefore, the hierarchical BEV pooling method reduces the amounts of CS cycles to $(1/H + 1/W)$ times that of the conventional methods. Given that BEV-SS typically adopts H and W values of 8 and 22, respectively, the BPC finally achieves $1.83\times$ higher throughput.

Fig. 5(a) illustrates the detailed architecture of the rasterization core with CS unit and BC unit, containing a CS array and BC circuits. The CS array comprises 22 CS processing elements (CSPEs) and conducts hierarchical width-wise CS for 16 channels of input camera features. Fig. 5(b) describes two operating modes of CSPE: accumulation mode and CS mode. In accumulation mode, parallel accumulation of input camera features according to height axis is performed with a register of each CSPE. In CS mode, serial width-wise cumulative sums across the registers of neighboring CSPEs are conducted. Meanwhile, the BCU figures out the grid boundaries through bitwise XOR operations on each 16-bit grid ID, followed by a 16-bit OR operation. The CS results and boundaries are temporarily stored in the CS buffer and boundary buffer, respectively, and only the boundary CS results are transmitted to external memory, as illustrated in Fig. 5(c), generating BEV pooling output for each grid. As a result, the hierarchical BPC design maximizes the computational parallelism for BEV pooling by transforming serial $N \times D \times W \times H$ operations into parallel $W + H$ operations, and further reduces the searching space of block-decomposed BEV pooling by 87.5%.

IV. COARSE-TO-FINE-GRAINED ZERO SKIPPING CONVOLUTION CLUSTER

A. Coarse-grained Zero Skipping through On-the-Fly Pruning

Redundant zero-computations, which account for 68% of the BEV encoder operations, can be skipped through sparsity

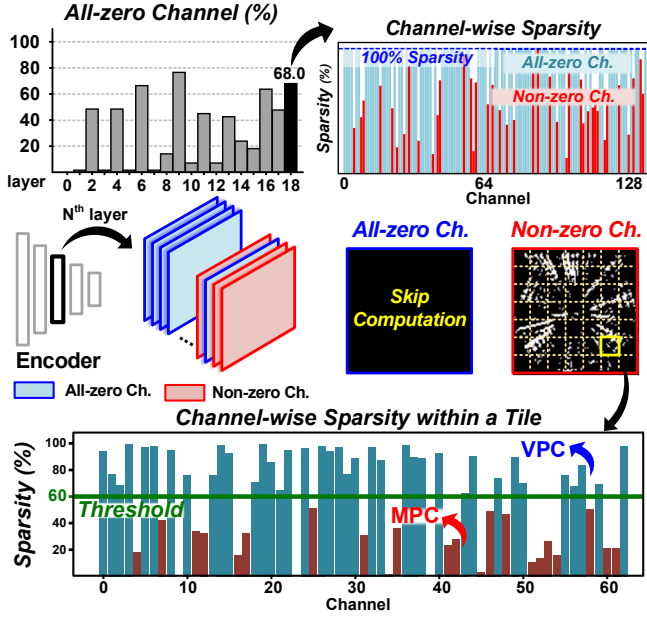


Fig. 6. Coarse-to-fine-grained zero skipping. On-the-fly pruning for coarse-grained zero skipping of meaningless channels, fine-grained zero skipping of non-zero channels on hybrid convolution cluster with sparsity-based core allocation.

exploitation to enhance the throughput. Fig. 6 illustrates the input sparsity characteristics of the BEV encoder. As depicted in the upper side of Fig. 6, meaningless channels, where all input features of a channel are zero, constitute 30.7% of the entire BEV encoder on average. Furthermore, 99.97% of all-zero channels maintain their sparsity across consecutive frames. To leverage the consecutive sparsity, on-the-fly pruning is introduced to skip the meaningless channels. Once an all-zero channel is detected, it is permanently excluded from computations in subsequent frames. Therefore, on-the-fly pruning method reduces the computational load by 50.1% and 52.8% on the Lyft Level 5 [9] and nuScenes [6] datasets, respectively, with only 0.3% and 0.08% drop in mean intersection over union (mIoU).

B. Tile-wise Fine-grained Zero Skipping on Hybrid Convolution Cluster

Sparsity remains in non-zero channels. The bottom of Fig. 6 illustrates the fluctuation in sparsity across the input channels within a 16×16 tile. To support the polarized sparsity, a hybrid CC inspired by [10] is utilized to separate operations based on the input sparsity. The hybrid CC consists of VPC, which adopts a sparsity exploitation architecture, and MPC, which employs a weight stationary design. As the computational efficiency of VPC declines when input sparsity falls below 60% [10], core allocation is determined based on a sparsity threshold of 60%. Sparse inputs are processed by the VPC for throughput increase by zero skipping. In contrast, dense inputs are processed by the MPC for memory access reduction through the enhanced reusability of the weight parameters.

Fig. 7 illustrates the hybrid sparse-dense CC. Input tiles are loaded from the shared I/O memory to VPC and MPC according to the sparsity of input data. Each core loads weight data from weight memory, operating with double-buffering. In

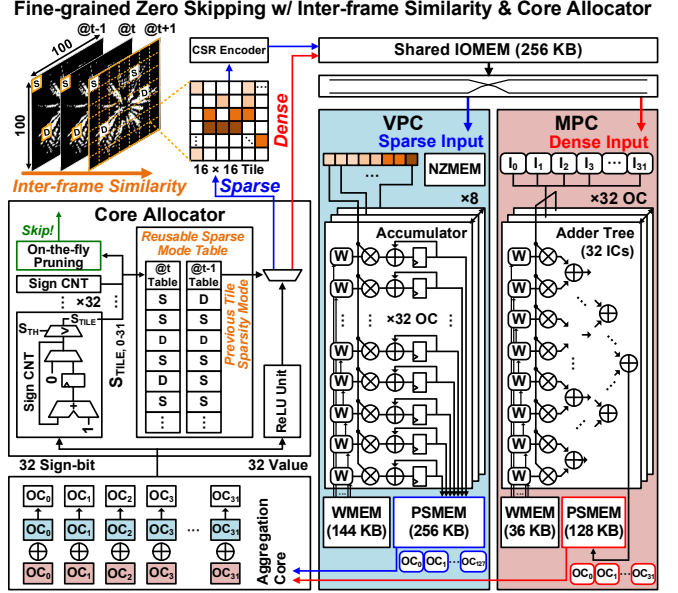
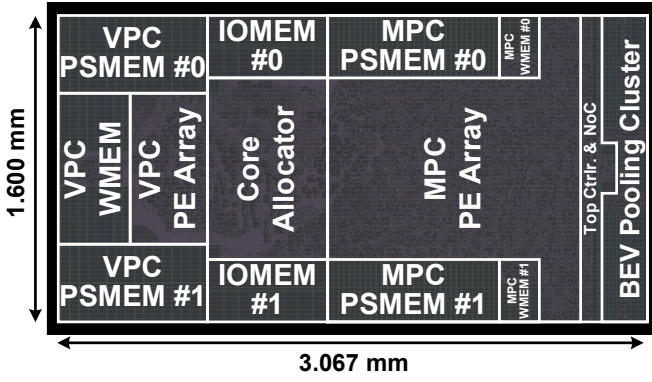


Fig. 7. The architecture of hybrid sparse-dense convolution cluster with VPC, MPC, aggregation core, and core allocator.

each cycle, the VPC loads eight non-zero inputs encoded through compressed sparse row (CSR) encoding. The CSR parameters, such as row pointer and column index, are loaded to non-zero memory for decoding. Then, it performs MAC operations with 32 weights to generate outputs for 32 output channels (OCs). These results are transferred to the internal partial-sum memory, where the results of PE arrays are accumulated. The MPC processes 32 input channels and 32 OCs each cycle. Then, the resulting 32 OCs from adder trees are accumulated in partial-sum memory. Once computations for a given output tile are done, the results from each core are combined through the aggregation core. Subsequently, a core allocator determines the processing core of the output tile for the next layer according to the sparsity.

Reference [10] proposed a sampling-based task allocation method for hybrid cores, utilizing the sparsity of the sampled pixels' inference result during MLP-based MAC operations for individual pixels. However, in CNNs, since the output features depend on all neighboring pixels within the receptive field, it is unsuitable for CNN-based BEV-SS. Therefore, exploiting sparsity in BEV-SS requires a data sparsity analysis after the computation for an output tile. It causes either memory overhead for temporary tile data storage or significant latency for input activation reordering. To address these challenges, the fine-grained zero skipping method by reusing inter-frame similarity-based sparsity is proposed.

Given the inter-frame similarity, 98.2% of the sparsity mode for each tile remains consistent across adjacent frames. By leveraging this consistency, a reusable sparsity mode table (RSMT) can be obtained from the output of the previous frame to assign each output tile to the appropriate core based on sparsity. The core allocator accumulates sign bit values from the results of the aggregation core prior to the ReLU unit, as negative values are set to zero after the ReLU activation function. Through the accumulated values, channel sparsity for each output tile is assessed, and the RSMT is generated for the



BEVSA (1 CC & 1 BPC)				
Technology	28nm CMOS			
Operating Condition	0.9 V, 500 MHz			
Area / Peak Power	4.91 mm ² / 877.6 mW			
On-chip SRAM Size	861 KB			
Core Type of CC	VPC		MPC	
# of MAC	256 / Core		1,024 / Core	
Input Activation Sparsity*	0%	90%	0%	90%
Performance*** [GOPS]	0% All-zero**	256	2,560	1,024
	40% All-zero	427	4,267	1,707
Power [mW]	0% All-zero	228.8	278.6	286.6
	40% All-zero	232.5	315.5	
Energy Efficiency [TOPS/W]	0% All-zero	1.12	9.19	3.57
	40% All-zero	1.84	13.5	5.96

*Sparsity within Tiles of BEV Encoder / **All-zero Channels of BEV Encoder
*** 1 MAC = 2 OP

Fig. 8. The photograph of the proposed BEVSA layout and the summary of its performance.

next frame. While the RSMT for the next frame is being generated, the values after the ReLU unit are allocated to either VPC or MPC for the following layer computations. Regardless of the sparsity calculation in the current frame, core allocation only considers the sparsity of the previous frame, referencing the RSMT. The sparse tiles assigned to the VPC are encoded through CSR encoding, with only the non-zero data stored in the shared I/O memory.

V. EVALUATION

A. Experimental Setup

CAD Tools. For the area and energy consumption evaluation, the proposed accelerator is designed in RTL Verilog and implemented by Synopsys Design Compiler for synthesis and the IC Compiler II for place and route in a 28nm CMOS technology process. The BEVSA is evaluated through cycle-accurate RTL simulation and power dissipation simulation using Synopsys PrimePower after the implementation.

Datasets and Model. To validate the throughput of the proposed BEVSA, the experiment is conducted on Lift-splat [2]. Two representative MCS datasets, nuScenes [6] and Lyft Level 5 [9], are used for evaluation in multifaceted environments with different camera settings. The pre-trained network models are sourced from TorchVision and EfficientNet-PyTorch.

Baseline Platform. For comparison with the previous platforms, Lift-splat is executed on an edge computing device, NVIDIA Jetson AGX Orin Developer Kit [7]. The evaluation

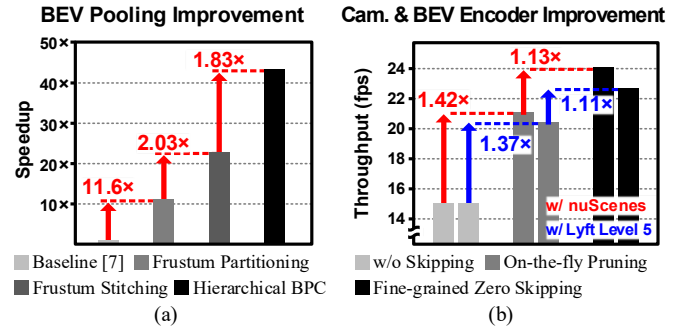


Fig. 9. Step-by-step performance improvement. (a) The speedup of BEV pooling, and (b) the throughput improvement of CNN-based encoders.

is conducted using the PyTorch [13] framework on 2048-core NVIDIA Ampere architecture GPU.

Implementation Results. Fig. 8 illustrates the post-layout of the proposed BEVSA and its performance summary. The accelerator occupies 4.91 mm² with a total of 861 KB on-chip SRAM and operates at a maximum frequency of 500 MHz at 0.9 V supply voltage, consuming 877.6 mW of peak power. The performance and energy efficiency are varied according to the sparsity of input activation in tile and the proportion of all-zero channels. The power consumption of the VPC slightly increases with higher input activation sparsity because its sparsity exploitation reduces accumulation latency, which results in more frequent partial-sum memory access during each unit operating time. However, the VPC significantly enhances performance at higher input activation sparsity by employing the proposed coarse-to-fine-grained zero skipping method. When comparing input activation sparsity ranging from 0% to 90%, the VPC improves energy efficiency by up to 8.21× despite a 21.8% increase in power consumption. Under conditions of 40% all-zero channels and 90% input activation sparsity within tiles, the VPC and MPC demonstrate peak performances of 4.267 and 1.707 TOPS with INT16 precision, respectively, achieving 12.1× and 1.67× higher energy efficiency than the non-sparsity condition.

B. Experimental Results

Improvement of BEV Pooling. Fig. 9(a) illustrates the step-by-step throughput improvement achieved through block-decomposed hierarchical BEV pooling. As discussed in Section III, the required searching space is lowered by 99.6% through the block-decomposed BEV pooling. After frustum partitioning, because of the decreased searching space and block-wise parallel computation, it improves the throughput by 11.6× compared to the baseline. Moreover, the overlapping regions between adjacent cameras are addressed by frustum stitching, achieving 2.03× higher throughput.

After the block decomposition, the hierarchical BPC facilitates parallel feature accumulation across height dimensions, further enhancing the pooling throughput by 1.83×. As a result, BEVSA with BPC achieves 43.2× higher BEV pooling throughput than the baseline. Furthermore, as the number of registers for the CS is diminished, the area of the CS array is decreased by 46.1% compared to 176 CSPEs for serial computation.

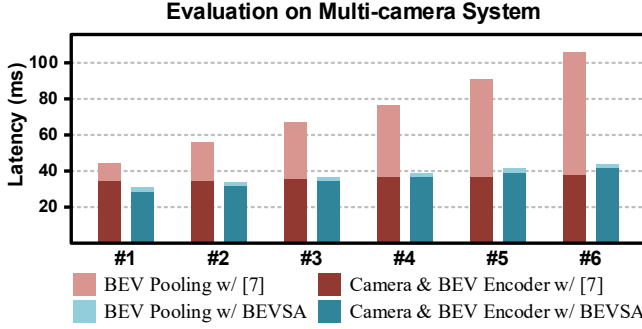


Fig. 10. The latency evaluation on multi-camera system with different number of camera images per frame.

Coarse-to-Fine-grained Zero Skipping. Fig. 9(b) illustrates the convolution throughput improvement by coarse-to-fine-grained zero skipping. The BEV features, input activation of the BEV encoder, are derived from depth estimation through the camera encoder and BEV pooling. The features exhibit high sparse data characteristic regardless of the camera rig, and the sparsity is maintained through the ReLU activation function. Specifically, the extracted BEV features with nuScenes and Lyft Level 5 datasets demonstrate high sparsity levels of 81.8% and 80.3%, which result in 69.1% and 45.7% of high average input activation sparsity in the BEV encoder, respectively. Therefore, almost 50% of computations of the whole BEV encoder are eliminated by on-the-fly pruning, and the throughput increases by 1.42 $\times$ and 1.37 $\times$ with each dataset, respectively. Subsequently, the fine-grained zero skipping through the hybrid CC further improves the throughput by 1.13 $\times$ and 1.11 $\times$, achieving 1.61 $\times$ and 1.52 $\times$ of the entire throughput improvement with nuScenes and Lyft datasets, respectively, validating the applicability across various camera rigs.

Impact of Number of Cameras. The BEV pooling significantly contributes to latency on the edge computing platform due to the sorting and irregular memory access. Therefore, the searching space of BEV pooling has a significant impact on the latency within [7]. Fig. 10 illustrates the latency evaluation for the proposed accelerator over the number of camera images. As the number of input camera images per frame increases, the latency in [7] grows substantially with the expanded pooling searching space. In contrast, the proposed BEVSA shrinks this searching space by 99.95%, thereby minimizing the impact of BEV pooling on overall latency. Although the latency in BEVSA also increases with the number of input images through the higher computational load in the camera encoder, the overall change accounts for 6.59% of total encoder operations, resulting in a significantly lower latency increase rate than [7]. Consequently, BEVSA supports more efficient processing in MCS with a larger number of input camera images compared to [7].

Total Performance. Table I shows the results of the comparison with conventional platforms for BEV-SS. BEVSA achieved 2.45 $\times$ throughput improvement compared to the edge computing platform [7], supporting 23.1 frames-per-second (fps) of real-time performance with 68.3 $\times$ reduced power consumption. Although the CPU + GPU platform [14] satisfies

TABLE I. PERFORMANCE COMPARISON TABLE

Device	# Cores	Technology	Area	Frequency	Throughput	Power	Energy-per-Frame
AGX Orin	12 CPU + 2,048 CUDA	8 nm	628.4 mm ²	1,300 MHz	9.43 fps	60 W	6,363 mJ/frame
Gold 6230+ RTX 2080 Ti	20 CPU + 4,352 CUDA	12 nm	754 mm ²	1,350 MHz	24.1 fps	250 W	10,373 mJ/frame
BEVSA w/ 1 CC	1,632 PEs	28 nm	4.91 mm ²	500 MHz	23.1 fps	0.878 W	38.01 mJ/frame
BEVSA w/ 2 CC	2,912 PEs	28 nm	9.34 mm ²	500 MHz	44.6 fps	1.664 W	37.31 mJ/frame

the real-time constraint, its high power consumption is unsuitable for edge computing. BEVSA improved the energy-per-frame by 167.4 $\times$ and 272.9 $\times$ compared to [7] and [14].

The proposed BEVSA minimizes the searching space of BEV pooling, making the overall latency mainly dependent on the camera and BEV encoder. Therefore, a range of hybrid CC configurations and clock frequencies can be implemented to meet specific application requirements. Increasing the number of CCs proportionally enhances throughput but incurs additional power and area overhead. For example, operating two CCs at a clock frequency of 500 MHz results in an additional power consumption of 0.786 W and an area overhead of 4.43 mm², while achieving a throughput of 44.6 fps. This performance meets the requirements of MCS as well as those of commercial cameras used in advanced autonomous driving systems ($\sim$ 36 Hz).

VI. CONCLUSION

In this paper, we present a real-time bird's-eye-view semantic segmentation accelerator (BEVSA) for 3D space perception in multi-camera system (MCS) with heterogeneous clusters. The block-decomposed hierarchical BEV pooling cluster reduces the searching space of sorting and irregular memory access by supporting inter- and intra-block parallel computations. It achieves a 43.2 $\times$ speedup for BEV pooling over the edge computing platform. In addition, the hybrid convolution cluster with coarse-to-fine-grained zero skipping method is proposed to improve the throughput for sparse convolution neural network (CNN) inference caused by BEV transformation. It skips meaningless zero computations by utilizing channel-wise on-the-fly pruning and tile-wise inter-frame sparsity similarity, achieving 1.61 $\times$ higher throughput on camera and BEV encoder. We evaluate the implemented BEVSA with 28nm CMOS technology compared to the edge computing platform by running two representative MCS datasets. As a result, the BEVSA achieved 23.1 frames-per-second of real-time BEV semantic segmentation throughput with 167.4 $\times$ higher energy-per-frame over the edge computing platform in the MCS.

ACKNOWLEDGMENTS

This work was supported in part by the National Research Foundation (NRF) of Korea grant funded by the Korea government (MSIT) (RS-2023-00212222); and in part by the Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2024-00399817). The EDA tool was supported by the IC Design Education Center (IDEC), Korea.

REFERENCES

- [1] H. Li, C. Sima, J. Dai, W. Wang, L. Lu and H. Wang, "Delving Into the Devils of Bird's-Eye-View Perception: A Review, Evaluation and Recipe," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 4, pp. 2151-2170, April 2024.
- [2] J. Philion and S. Fidler, "Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d," in *Proc. of the European Conference on Computer Vision (ECCV)*, 2020, pp. 194-210.
- [3] X. Gu, G. Song, I. Gilitschenski, M. Pavone and B. Ivanovic, "Producing and Leveraging Online Map Uncertainty in Trajectory Prediction," in *Proc. of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 14521-14530.
- [4] S. Hu, L. Chen, P. Wu, H. Li, J. Yan and D. Tao, "ST-P3: End-to-end vision-based autonomous driving via spatial-temporal feature learning," in *Proc. of the European Conference on Computer Vision (ECCV)*, 2022, pp. 533-549.
- [5] C. Ge, J. Chen, E. Xie, Z. Wang, L. Hong, H. Lu, Z. Li and P. Luo, "MetaBEV: Solving Sensor Failures for 3D Detection and Map Segmentation," in *Proc. of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 8721-8731.
- [6] H. Caesar, V. Bankiti, A. -H. Lang, S. Vora, V. -E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan and O. Beijbom, "nuScenes: A Multimodal Dataset for Autonomous Driving," in *Proc. of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11621-11631.
- [7] NVIDIA. Jetson Orin Series. Accessed: Nov, 7, 2024. [Online]. Available: <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>
- [8] X. Feng, W. Sun, X. Lin, S. Fan, H. Yang and Y. Liu, "A 28nm 1.2GHz 5.27TOPS/W Scalable Vision/Point Cloud Deep Fusion Processor with CAM-based Universal Mapping Unit for BEVFusion Applications," in *Proc. of IEEE Custom Integrated Circuits Conference (CICC)*, 2024, pp. 1-2.
- [9] J. Houston, G. Zuidhof, L. Bergamini, Y. Ye, L. Chen, A. Jain, S. Omari, V. Iglovikov and P. Ondruska, "One Thousand and One Hours: Self-driving Motion Prediction Dataset," in *Proc. of Conference on Robot Learning*, 2020, pp. 409-418.
- [10] D. Han, J. Ryu, S. Kim, S. Kim, J. Park and H. -J. Yoo, "MetaVRain: A Mobile Neural 3-D Rendering Processor With Bundle-Frame-Familiarity-Based NeRF Acceleration and Hybrid DNN Computing," *IEEE Journal of Solid-State Circuits*, vol. 59, no. 1, pp. 65-78, Jan. 2024.
- [11] D. Im, G. Park, J. Ryu, Z. Li, S. Kang and H. -J. Yoo, "DSPU: An Efficient Deep Learning-Based Dense RGB-D Data Acquisition With Sensor Fusion and 3-D Perception SoC," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 1, pp. 177-188, Jan. 2023.
- [12] J. Jung, S. Kim, B. Seo, W. Jang, S. Lee, J. Shin, D. Han and K. J. Lee, "An Energy-Efficient Processor for Real-Time Semantic LiDAR SLAM in Mobile Robots," *IEEE Journal of Solid-State Circuits*, early access, Sep. 06, 2024, doi: 10.1109/JSSC.2024.3450314.
- [13] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. of International Conference on Neural Information Processing Systems (NIPS)*, 2019, pp. 8026-8037.
- [14] NVIDIA. (2080). GeForce RTX 2080 Ti. Accessed: Nov, 7, 2024. [Online]. Available: <https://www.nvidia.com/en-us/geforce/20-series/>